HDOC DAY-64

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array contains exactly one element repeated once. Pass the array to a function named duplicate_element() using its address. Use pointers to pass the array between the calling function and duplicate_element(). The function duplicate_element() must:

- Find the only repeated element in nums[] and display the repeated element.
- Find the result in a single iteration through the array by using an efficient approach such as the XOR technique.
- Assume that every other element appears exactly once.

Example 1:

Array: 1 3 4 2 2

Output:

Repeated element: 2

Example 2:

Array: 3 1 3 4 2

Output:

Repeated element: 3

Example 3:

Array: 1 4 5 2 3 4

Output:

Repeated element: 4

Algorithm:

1. Start.
2. Read the size n of the array.
3. Declare the variable-length array nums[n].

4. Read n elements into the array.
5. Pass the address of the array to duplicate_element().
6. Initialize result = 0.
7. XOR all elements of nums[].
8. XOR result with every number from 1 to n-1.
9. Because x ^ x = 0, all non-repeated elements cancel out.
10. Return the remaining value, which is the repeated element.
11. Display the repeated element.
12. Stop.

- **C Program:**

```
#include <stdio.h>

int duplicate_element(int *nums, int size)
{
    int result = 0;

    // XOR all array elements
    for (int i = 0; i < size; i++)
    {
        result ^= nums[i];
    }

    // XOR numbers from 1 to size-1
    for (int i = 1; i < size; i++)
    {
        result ^= i;
    }

    return result;
}

int main()
{
    int size;

```

```
    printf("Enter size of array: ");
    scanf("%d", &size);

    int nums[size];   // Variable-Length Array (VLA)

    printf("Enter array elements: ");
    for (int i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    int repeated = duplicate_element(nums, size);

    printf("Repeated element: %d\n", repeated);

    return 0;
}
```

Test Case Table:

Test Case	Size	Input array	Expected Output
1	5	1 3 4 2 2	Repeated Element: 2
2	5	3 1 3 4 2	Repeated Element: 3
3	6	1 4 5 2 3 4	Repeated Element: 4
4	6	5 1 2 3 4 5	Repeated Element: 1
5	3	1 2 1	Repeated Element: 1

Compilation and Execution:

The following test cases has been compiled and executed successfully.

Test Case 1:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 5
Enter array elements: 1 3 4 2 2
Repeated element: 2
```

Test Case 2:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 5
Enter array elements: 3 1 3 4 2
Repeated element: 3
```

Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 6
Enter array elements: 1 4 5 2 3 4
Repeated element: 4
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 6
Enter array elements: 5 1 2 3 4 5
Repeated element: 5
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 3
Enter array elements: 1 2 1
Repeated element: 1
```